

Lecture 17. Ch. 12. Generative deep learning A

12.1 Text generation

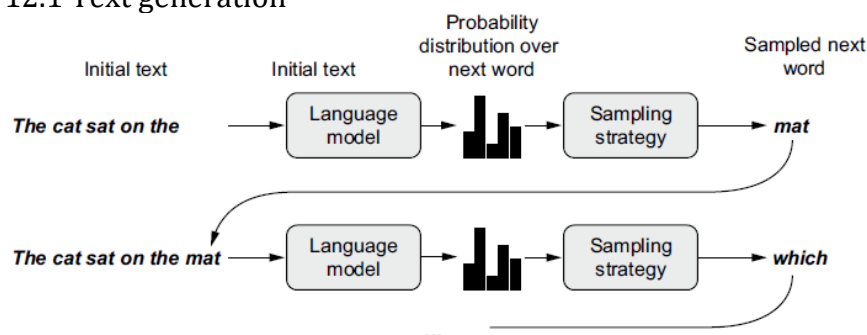


Figure 12.1 The process of word-by-word text generation using a language model

Sampling = selecting a word

Greedy: Select a word with the highest probability. This is deterministic. [boring; not creative.]

Random: Randomly draw a word from a pool according to each word's likelihood. [Surprising]

The randomness can be controlled by a parameter 'temperature.'

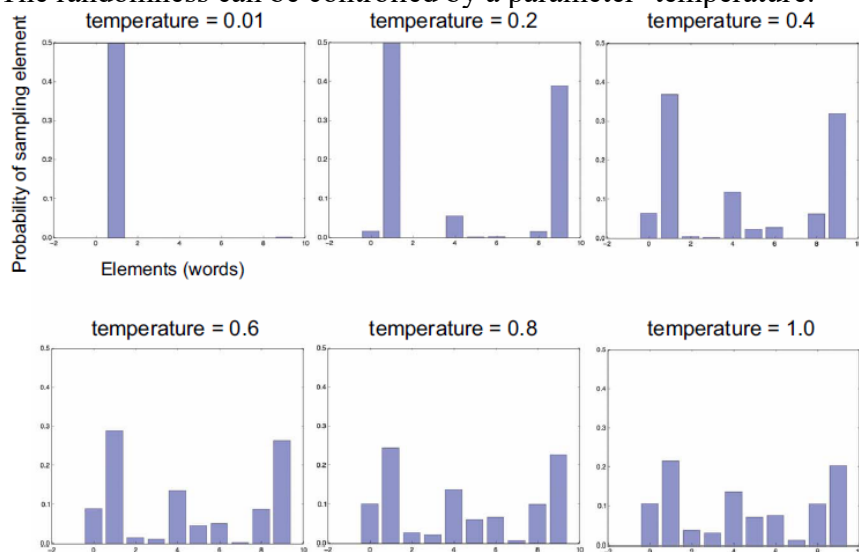


Figure 12.2 Different reweightings of one probability distribution. Low temperature = more deterministic, high temperature = more random.

Examples:

- With temperature=0.2
 - “this movie is a [UNK] of the original movie and the first half hour of the movie is pretty good but it is a very good movie it is a good movie for the time period”
- With temperature=0.5
 - “this movie is a waste of time and money i have to say that this movie was a complete waste of time i was surprised to see that the movie was made up of a good movie and the movie was not very good but it was a waste of time and”

- With `temperature=0.7`
 - “this movie is fun to watch and it is really funny to watch all the characters are extremely hilarious also the cat is a bit like a [UNK] [UNK] and a hat [UNK] the rules of the movie can be told in another scene saves it from being in the back of”
- With `temperature=1.0`
 - “this movie was a masterpiece away from the storyline but this movie was simply exciting and frustrating it really entertains friends like this the actors in this movie try to go straight from the sub thats image and they make it a really good tv show”
- With `temperature=1.5`
 - “this movie could be so unbelievably lucas himself bringing our country wildly funny things has is for the garish serious and strong performances colin writing more detailed dominated but before and that images gears burning the plate patriotism we you expected dyan bosses devotion to must do your own duty and another”

12.3 Neural style transfer

Use a pre-trained network
 Need two input images, one for content, the other for style
 Training is not “traditional”

Load Pretrained VGG19
 Remove the fully connected layers
 Keep only convolutional layers
 Freeze weights

Choose Layers for Content & Style
 Early layers → textures
 Deeper layers → structure

Define Loss Functions

Optimize the Image
 Start with a content image
 Treat the image as a parameter
 Use gradient descent to update pixels

Iterate
 Forward pass through VGG19
 Compute losses
 Backpropagate to update image
 Repeat (hundreds–thousands of steps)



Figure 12.9 A style transfer example
 (using *Starry Night* by Vincent Van Gogh)

Train what? → The variables are the **pixels** in the **combination image**

```
loss = distance( style(reference_image) - style(combination_image))
        + distance(content(original_image) - content(combination_image))
```

Set up a network that computes VGG19 layer activations for the style-reference image, the base image, and the generated image at the same time.

Getting the style and content images

Content = features or activations (X), at one later layer.
 Style = Gram matrix of the features (XX^T), at multiple layers.

```
from tensorflow import keras
base_image_path = keras.utils.get_file(
    "sf.jpg", origin="https://img-datasets.s3.amazonaws.com/sf.jpg")
style_reference_image_path = keras.utils.get_file(
    "starry_night.jpg",
    origin="https://img-datasets.s3.amazonaws.com/starry_night.jpg")
original_width, original_height = keras.utils.load_img(base_image_path).size
img_height = 400
img_width = round(original_width * img_height / original_height)
```

Auxiliary functions

```
import numpy as np
def preprocess_image(image_path):
    img = keras.utils.load_img(
        image_path, target_size=(img_height, img_width))
    img = keras.utils.img_to_array(img)
    img = np.expand_dims(img, axis=0)
    img = keras.applications.vgg19.preprocess_input(img)
    return img
def deprocess_image(img):
    img = img.reshape((img_height, img_width, 3))
    img[:, :, 0] += 103.939
    img[:, :, 1] += 116.779
    img[:, :, 2] += 123.68
    img = img[:, :, ::-1]
    img = np.clip(img, 0, 255).astype("uint8")
    return img
```

Using a pretrained VGG19 model to create a feature extractor

```
# Build a VGG19 model loaded with pretrained ImageNet weights.
model = keras.applications.vgg19.VGG19(weights="imagenet", include_top=False)
outputs_dict = dict([(layer.name, layer.output) for layer in model.layers])
```

"dict" is actually "feature"

```
# Model that returns the activation values for every target layer (as a dict)
feature_extractor = keras.Model(inputs=model.inputs, outputs=outputs_dict)
```

Content loss

```
def content_loss(base_img, combination_img):
    return tf.reduce_sum(tf.square(combination_img - base_img))
```

Style loss

```
def gram_matrix(x):
    x = tf.transpose(x, (2, 0, 1))
    features = tf.reshape(x, (tf.shape(x)[0], -1))
    gram = tf.matmul(features, tf.transpose(features))
    return gram

def style_loss(style_img, combination_img):
    S = gram_matrix(style_img)
    C = gram_matrix(combination_img)
    channels = 3
    size = img_height * img_width
    return tf.reduce_sum(tf.square(S - C)) / (4.0 * (channels ** 2) * (size ** 2))
```

a 3rd loss: **Total variation (TV) loss** (enforcing a piecewise-constant final image)

```
def total_variation_loss(x):
    a = tf.square(
        x[:, : img_height - 1, : img_width - 1, :] - x[:, 1:, : img_width - 1, :]
    )
    b = tf.square(
        x[:, : img_height - 1, : img_width - 1, :] - x[:, : img_height - 1, 1:, :]
    )
    return tf.reduce_sum(tf.pow(a + b, 1.25))
```

Defining the final loss that you'll minimize

```
# List of layers to use for the style loss
style_layer_names = [
    "block1_conv1",
    "block2_conv1",
    "block3_conv1",
    "block4_conv1",
    "block5_conv1",
]

# The layer to use for the content loss
content_layer_name = "block5_conv2"

# Contribution weight of the total variation loss
total_variation_weight = 1e-6

# Contribution weight of the style loss
style_weight = 1e-6

# Contribution weight of the content loss
content_weight = 2.5e-8

def compute_loss(combination_image, base_image, style_reference_image):
    input_tensor = tf.concat(
        [base_image, style_reference_image, combination_image], axis=0)
    features = feature_extractor(input_tensor)
    loss = tf.zeros(shape=()) # Initialize the loss to 0.
    # Add the content loss.
    layer_features = features[content_layer_name]
    base_image_features = layer_features[0, :, :, :]
    combination_features = layer_features[2, :, :, :]
    loss = loss + content_weight * content_loss(
        base_image_features, combination_features
    )
    # Add the style loss.
    for layer_name in style_layer_names:
        layer_features = features[layer_name]
        style_reference_features = layer_features[1, :, :, :]
        combination_features = layer_features[2, :, :, :]
        style_loss_value = style_loss(
            style_reference_features, combination_features
        )
        loss += (style_weight / len(style_layer_names)) * style_loss_value
```

```
# Add the total variation loss.
loss += total_variation_weight * total_variation_loss(combination_image)
return loss
```

Setting up the gradient-descent process

```
import tensorflow as tf
```

We make the training step fast by compiling it as a tf.function.

```
@tf.function
```

```
def compute_loss_and_grads(
    combination_image, base_image, style_reference_image):
    with tf.GradientTape() as tape:
        loss = compute_loss(
            combination_image, base_image, style_reference_image)
        grads = tape.gradient(loss, combination_image)
    return loss, grads
```

We'll start with a learning rate of 100 and decrease it by 4% every 100 steps.

```
optimizer = keras.optimizers.SGD(keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate=100.0, decay_steps=100, decay_rate=0.96
))
```

```
base_image = preprocess_image(base_image_path)
```

```
style_reference_image = preprocess_image(style_reference_image_path)
```

Use a Variable to store the combination image since we'll be updating it during training.

```
combination_image = tf.Variable(preprocess_image(base_image_path))
iterations = 4000
```

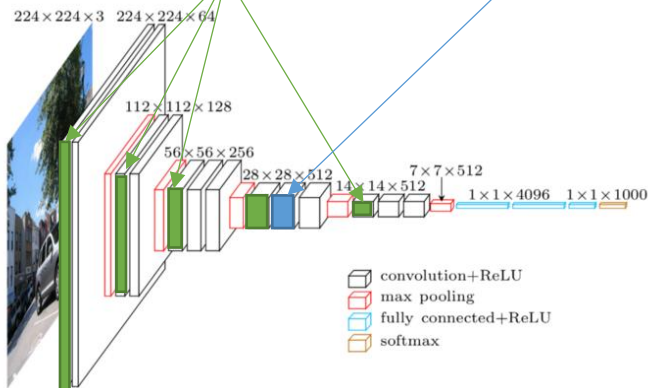
```
for i in range(1, iterations + 1):
    loss, grads = compute_loss_and_grads(
        combination_image, base_image, style_reference_image
    )
    optimizer.apply_gradients([(grads, combination_image)])
    if i % 100 == 0:
        print(f"Iteration {i}: loss={loss:.2f}")
        img = deprocess_image(combination_image.numpy())
        fname = f"combination_image_at_iteration_{i}.png"
        # Save the combination image at regular intervals.
        keras.utils.save_img(fname, img)
```

VGG19 (main application: classification)

- Conv3x3 (64)
- Conv3x3 (64)
- MaxPool
- Conv3x3 (128)
- Conv3x3 (128)
- MaxPool
- Conv3x3 (256)
- Conv3x3 (256)
- Conv3x3 (256)
- MaxPool
- Conv3x3 (512)
- Conv3x3 (512)
- Conv3x3 (512)
- Conv3x3 (512)
- MaxPool
- Conv3x3 (512)
- Conv3x3 (512)
- Conv3x3 (512)
- MaxPool
- Conv3x3 (512)
- Conv3x3 (512)
- MaxPool
- Fully Connected (4096)
- Fully Connected (4096)

- Fully Connected (1000)
- SoftMax

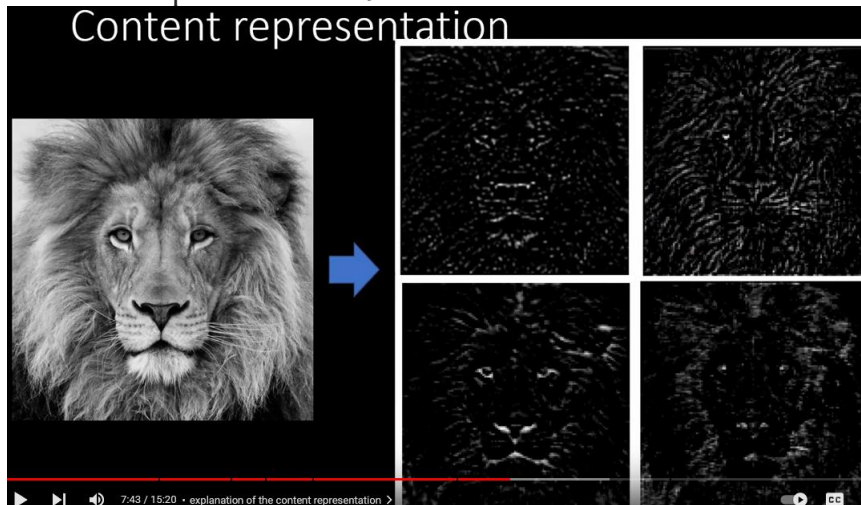
Style layers are in green. Content layer is in blue.



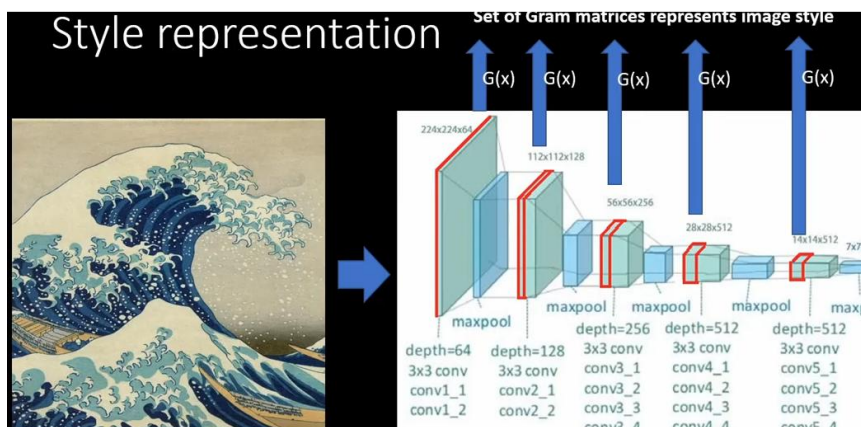
This VGG19 model is used three times:

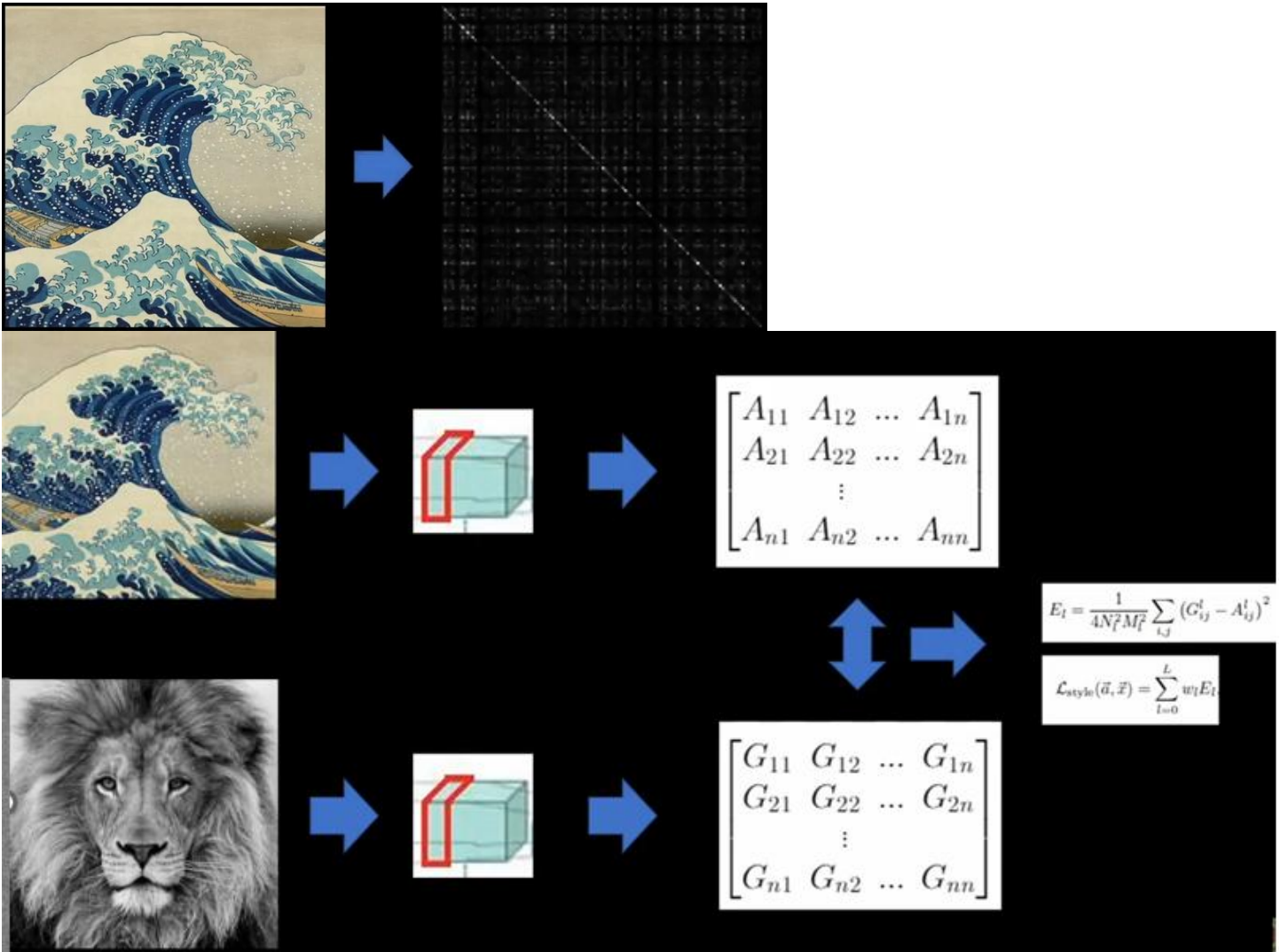
- (1) the base-image as input to generate “contents”;
- (2) the style-image as input to generate “styles”;
- (3) the combination-image as input to generate both “contents” and “styles”.

The final output of the VGG19 is not used.



'Content' is simply the 'activations'





<https://www.youtube.com/watch?v=B22nIUhXo4E>

'Style': Flatten out the activations and find outer product. Then drive G to A.

The **content** of an image is represented by the values of the intermediate feature maps.

The **style** of an image can be described by the means and correlations across the different feature maps. Calculate a **Gram matrix** that includes this information by taking the **outer product** of the feature vector with itself at each location and averaging that outer product over all locations. This Gram matrix can be calculated for a particular layer as:

$$G_{cd}^l = \frac{\sum_{ij} F_{ijc}^l(x) F_{ijd}^l(x)}{IJ}$$

'Parameters' to be 'trained' = the output combined image

The VGG19 convolutional neural network is pre-trained and fixed.

